

Guía de Estudio Intensiva: Desarrollo Móvil Multiplataforma (Móviles II)

Autor: Catedrático de Ingeniería de Software

Objetivo: Preparación para examen teórico de arquitectura y patrones.

1. Tabla Comparativa de Arquitecturas (El "Santo Grial" del examen)

Entender esta tabla es garantizar el 40% de la nota. Los profesores preguntan sobre diferencias de rendimiento y renderizado.

Característica	Ionic (+ Angular)	React Native	Flutter
Tipo de App	Híbrida (WebView)	Nativa Cross-Platform (Bridge/JSI)	Nativa Cross-Platform (Canvas)
Lenguaje Base	TypeScript / HTML / SCSS	JavaScript / TypeScript	Dart
Motor de Renderizado	WebView (Navegador empotrado). Usa DOM.	Componentes Nativos. Mapea JS a vistas nativas (UIView/AndroidView).	Skia / Impeller. Dibuja cada píxel en un Canvas propio.
Acceso Nativo	A través de Capacitor o Cordova (Plugins).	A través de Bridge o TurboModules (JSI).	A través de Platform Channels (Binary Messenger).
Rendimiento	Limitado por el WebView. Bueno para CRUDs, malo para gráficos intensivos.	Cercano al nativo, pero el "Bridge" puede ser cuello de botella en animaciones.	Excelente (60-120 fps). Código compilado a nativo ARM (AOT).
Filosofía UI	"Write once, run anywhere" (La misma UI web en	"Learn once, write anywhere" (Usa componentes	"Pixel Perfect" (Control total, la UI es idéntica en

	todos lados).	nativos, la UI se adapta al OS).	todos lados).
--	---------------	----------------------------------	---------------

2. Conceptos Clave por Tecnología (Lo que DEBES saber)

A. Angular + Ionic

1. **Inyección de Dependencias (DI):** Es el corazón de Angular. Debes saber explicar cómo los *Services* se inyectan en los componentes para desacoplar la lógica de la vista.
2. **Observables (RxJS) vs Promesas:** Pregunta clásica. Las Promesas manejan un solo evento asíncrono; los Observables son flujos de datos continuos (streams) que se pueden cancelar y transformar.
3. **Ciclo de Vida (Lifecycle Hooks):** Memoriza `ngOnInit` (inicialización lógica), `ngAfterViewInit` (DOM listo) y `ngOnDestroy` (limpieza de memoria/suscripciones).
4. **Shadow DOM:** Ionic usa Web Components. El Shadow DOM encapsula los estilos y el marcado para que el CSS global no rompa tus componentes.
5. **Capacitor:** Entiende que es la capa que comunica el JavaScript del WebView con el código Nativo (Cámara, GPS) mediante promesas.

B. React Native

1. **The Bridge (Arquitectura Vieja) vs JSI (Arquitectura Nueva):**
 - *Bridge:* Serializa mensajes JSON entre el hilo JS y el hilo Nativo. Es lento y asíncrono.
 - *JSI (JavaScript Interface):* Permite a JS invocar funciones nativas C++ directamente (síncrono).
2. **Virtual DOM:** React mantiene una copia ligera del DOM en memoria. Calcula diferencias (diffing) y solo actualiza lo necesario en la UI nativa.
3. **Hilos (Threading Model):**
 - *Main Thread (UI):* Dibuja la interfaz nativa.
 - *JS Thread:* Ejecuta tu código React. Si este se bloquea, la app se congela.
4. **JSX:** No es HTML, es azúcar sintáctica para crear objetos JavaScript que representan UI.
5. **Hooks:** `useEffect` (manejo de ciclo de vida y side-effects) y `useState` (manejo de estado local).

C. Flutter

1. **Todo es un Widget:** No hay separación entre vista y controlador clásica. La UI se define declarativamente.
2. **Árboles de Flutter (The 3 Trees):** Pregunta avanzada (nivel "Matrícula de Honor").
 - *Widget Tree:* La configuración (lo que escribes).
 - *Element Tree:* La instanciación (maneja el ciclo de vida y estado).
 - *RenderObject Tree:* El encargado de pintar y calcular tamaños (layout).

3. JIT vs AOT:

- *Desarrollo*: Usa JIT (Just In Time) para permitir el **Hot Reload** (inyectar código nuevo sin perder estado).
- *Producción*: Usa AOT (Ahead Of Time) compilando a código máquina ARM para velocidad pura.

4. State Management: Entiende la diferencia entre Estado Efímero (setState, local en un widget) y Estado de App (Bloc, Provider, Riverpod).

5. BuildContext: Es la referencia de dónde está un widget en el árbol de elementos. Sin él, no sabes "dónde estás".

3. Simulacro de Examen (Preguntas "Trampa" y Respuestas)

P1: En términos de renderizado, ¿cuál es la diferencia fundamental entre una lista infinita en Ionic y una en Flutter?

- **R:** En **Ionic**, cada elemento es un nodo DOM en un WebView; demasiados nodos consumen mucha memoria y ralentizan el scroll. En **Flutter**, se usa un motor gráfico (Skia) que solo dibuja los píxeles visibles en un Canvas; no existen "objetos" nativos pesados, lo que lo hace mucho más performante en listas largas.

P2: ¿Por qué se dice que el "Bridge" de React Native es un cuello de botella?

- **R:** Porque la comunicación entre el hilo de JavaScript (lógica) y el hilo Nativo (UI) requiere serializar datos a JSON y enviarlos de forma asíncrona. Si hay muchos eventos (ej: scroll rápido o animaciones complejas), el puente se satura y la UI pierde frames (se traba).

P3: Explique la diferencia entre Hot Reload y Hot Restart en Flutter.

- **R:** **Hot Reload** inyecta los nuevos archivos de código fuente en la Máquina Virtual Dart en ejecución manteniendo el estado (variables). **Hot Restart** reinicia la aplicación completamente, perdiendo el estado, y reconstruye el árbol de widgets desde cero.

P4: En Angular/Ionic, ¿para qué sirve el archivo module.ts (o el concepto de Standalone Components en versiones nuevas)?

- **R:** Sirve para organizar la compilación y la Inyección de Dependencias. Declara qué componentes pertenecen a ese bloque, qué servicios provee y qué librerías externas importa. En versiones nuevas, los *Standalone Components* eliminan la necesidad de Módulos para simplificar la arquitectura.

P5: ¿Qué problema resuelve Redux, NgRx o Bloc en una arquitectura móvil?

- **R:** Resuelven el problema del **Estado Compartido** y la **Predictibilidad**. Evitan el "Prop Drilling" (pasar datos de padre a hijo a nieto...) y centralizan la verdad de la aplicación en un único Store inmutable, facilitando el debugging.

P6: Si usted necesita una App que utilice funciones nativas muy específicas (ej: drivers USB propietarios) y requiere máximo rendimiento de UI, ¿qué tecnología descartaría

primero y por qué?

- **R:** Descartaría **ionic**. Al correr en un WebView, añadir soporte para drivers propietarios complejos es más difícil (requiere plugins de Capacitor complejos) y el rendimiento de UI siempre será inferior al nativo o compilado.

P7: ¿Qué es el "Shadow DOM" y por qué es relevante en Ionic?

- **R:** Es un estándar web que permite encapsular un sub-árbol del DOM. Ionic lo usa para que sus componentes (botones, cards) tengan estilos protegidos que no se vean afectados por el CSS global de la aplicación del usuario.

P8: Describa el ciclo de vida de un componente React (Functional Component) usando Hooks.

- **R:**
 1. **Montaje (Mount):** Se ejecuta el cuerpo de la función y luego `useEffect(() => {}, [])`.
 2. **Actualización (Update):** Se re-ejecuta al cambiar props o estado, seguido de `useEffect(() => {}, [deps])`.
 3. **Desmontaje (Unmount):** Se ejecuta la función de retorno (cleanup) dentro del `useEffect`.

P9: ¿Por qué Flutter no utiliza los widgets nativos (OEM) del sistema operativo (como un UIButton de iOS)?

- **R:** Para garantizar consistencia visual absoluta ("Pixel Perfect"). Al dibujar sus propios widgets, Flutter evita problemas de fragmentación donde una app se ve diferente en distintas versiones de Android o iOS.

P10: ¿Cuál es la diferencia entre una PWA (Progressive Web App) y una App Híbrida (Ionic/Capacitor)?

- **R:** Una **PWA** es una web que se ejecuta en el navegador del usuario (sin acceso total al hardware). Una **App Híbrida** se empaqueta como una app nativa (.apk / .ipa), se descarga de la tienda, y tiene acceso al hardware nativo a través de un puente (Capacitor/Cordova), aunque visualmente se renderice vía web.

4. Diferencias Arquitectónicas (Resumen Ejecutivo)

Utiliza estos términos para sonar profesional:

1. **Hybrid Wrappers (Ionic):**
 - *Arquitectura:* Una página web (index.html) corriendo dentro de un navegador sin barra de direcciones (WebView).
 - *Clave:* "El navegador es el sistema operativo".
2. **Native Scripting / Bridges (React Native):**
 - *Arquitectura:* Lógica en JS gestionando componentes nativos reales mediante un puente de comunicación.
 - *Clave:* "Lógica interpretada, UI Nativa".

3. **Custom Rendering / Compiled (Flutter):**

- *Arquitectura:* Un motor de videojuegos 2D (Skia) controla la pantalla completa. Se salta las jerarquías de vistas del sistema operativo.
- *Clave:* "Dueño de cada píxel".